

Отчёт по лабораторной работе

Файл: labA_01.cpp

Н.А. Пономарев, группа 25.M71-мм

22 января 2026 г.

Некоторые обозначения, используемые в данном тексте:

- $A_{m \times n} = (a_{ij})$ — матрица из m строк и n столбцов, а a_{ij} — элемент матрицы на строке i , столбце j .
- $\mathbb{1}_{m \times n}$ — матрица из единиц, соответствующего размера.
- $E_{n \times n}$ — единичная матрица.
- δ_{ij} — символ Кронекера.

1. Алгоритм

Алгоритм вычисляет значение некоторого матричного выражения. Все элементы матриц принадлежат полю вещественных чисел. На вход алгоритм получает матрицы $B_{n \times n}$, $C_{n \times n}$, и векторы $x_{n \times 1}$, $y_{n \times 1}$. В результате работы алгоритма получается матрица $A_{n \times n}$.

Первым делом вычисляется значение скаляра E_x по формуле:

$$E_x = \sum_{i=0}^{n-1} x_i = \mathbb{1}_{1 \times n} \cdot x_{n \times 1}.$$

Фактически считаем скалярное произведение.

Затем вычисляется вектор $CE_{n \times 1}$:

$$CE = C_{n \times n} \cdot \mathbb{1}_{n \times 1}$$

или поэлементно:

$$ce_{i1} = \sum_{k=0}^{n-1} c_{ik}.$$

Фактически получаем вектор сумм строк матрицы C .

Далее вычисляем скаляр trace , который не является настоящим следом матрицы:

$$\text{trace} = \sum_{i=0}^{n-1} \sum_{k=0}^{n-1} b_{ik} \cdot c_{k1} = \mathbb{1}_{1 \times n} \cdot B_{n \times n} \cdot C E_{n \times 1}.$$

Затем вычисляем скаляр z_1 :

$$z_1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} b_{ij} \cdot E x \cdot y_i = E x \cdot (y^\top)_{1 \times n} \cdot B_{n \times n} \cdot \mathbb{1}_{n \times 1},$$

скаляр z_2 (просто скалярное произведение):

$$z_2 = \sum_{i=0}^{n-1} x_i y_i = (x^\top)_{1 \times n} \cdot y_{n \times 1},$$

и скаляр $z = z_1/z_2$.

В конце формируем матрицу $A_{n \times n}$:

$$A = \text{trace} \cdot C_{n \times n} + z \cdot \mathbb{1}_{n \times n} + E_{n \times n}$$

или поэлементно:

$$a_{ij} = \text{trace} \cdot c_{ij} + z + \delta_{ij}.$$

Обсуждение алгоритма. Сам по себе алгоритм большую часть времени занимается свёртками и выглядит довольно сложным для распараллеливания, поскольку мало независимых операций и вероятно большая нагрузка на работу с памятью.

2. Работа с кодом

В процессе выполнения работы код был модифицирован следующим образом:

- Исправлена синтаксическая ошибка в виде незакрытой фигурной скобки.
- Добавлен парсер аргументов командной строки.
- Слегка изменён вывод для упрощения постановки экспериментов.
- Декомпозирована логика вычислений для упрощения исследования и распараллеливания.
- Добавлен CMAKE.
- Применены исправления от CLANG-FORMAT и CLANG-TIDY.
- Написан скрипт для проведения замеров.

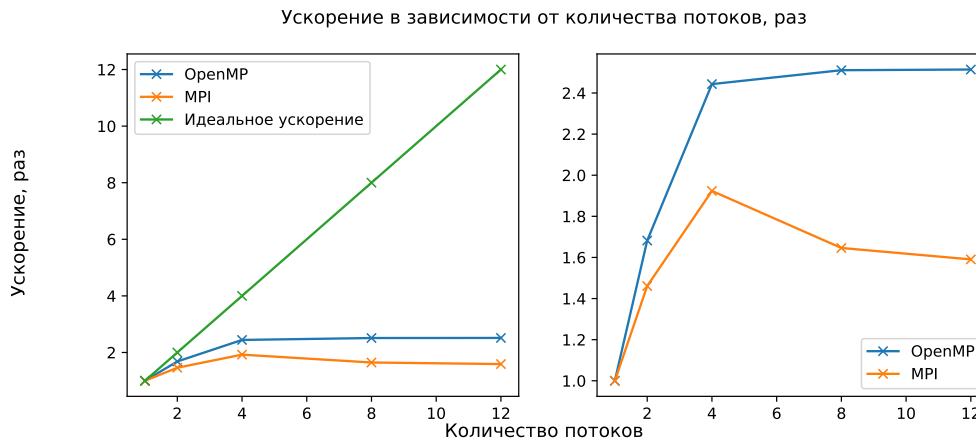


Рис. 2

5. Исследование масштабируемости

Измерения проводились на системе с процессором Intel(R) Xeon(R) E-2136 CPU @ 3.30GHz с 12 потоками и 64 ГиБ оперативной памяти, с ОС UBUNTU 22.04, компилятором GCC 10.2.0 и MPICH 4.1.1.

В случае с MPI измерения на кластере произвести не удалось, поэтому кластером служил тот же многоядерный процессор.

Измерения проводились при фиксированном $n = 10\,000$ и изменяющемся количестве потоков от 1 до максимальной степени двойки меньшей количеству потоков и с использованием всех потоков.

Перед запуском измерений при некотором фиксированном количестве потоков производилось 3 разогревочных запуска, затем производилось 10 измерений.

Итого, были получены результаты, изображенные на рисунке 2 и в таблице 1. Графики идентичны друг другу, отличие только в том, что на правом отсутствует график идеального распределения ради более детального анализа фактического ускорения.

Анализ ускорения. График показывает, что достичь идеального ускорения в соответствии с законом Амдала даже близко не получается. Более того, в сравнении с 4 потоками, на 8 и 12 потоках в случае OPENMP прирост почти не наблюдается, а в случае MPI происходит замедление. Полагаю, что это связано с тем, что затраты на обмен данными и создание потоков начинают сравниться с временем вычислений для OPENMP и превышают затраты на вычисления в случае MPI.

Выводы

К сожалению, большого ускорения достичь не удалось. На небольшом числе потоков (2 и 4) ускорение было хоть сколько-то близко к идеальному ускорению в

№ итерации	Время исполнения алгоритма при $n = 10\,000$, с									
	OpenMP					MPI				
Кол-во потоков	1	2	4	8	12	1	2	4	8	12
0	372	221	153	148	148	419	288	218	258	265
1	372	222	152	149	149	418	288	218	255	264
2	375	222	152	149	148	420	287	218	254	264
3	372	221	153	148	148	419	287	219	257	264
4	373	221	152	149	148	419	288	218	255	264
5	372	222	153	148	148	420	288	218	254	263
6	373	221	152	148	149	422	286	220	255	265
7	372	222	152	148	148	423	287	218	255	264
8	373	221	153	148	148	419	288	217	253	262
9	372	222	153	149	148	420	287	219	255	265
Ускорение, раз	1.00	1.68	2.44	2.51	2.51	1.00	1.46	1.92	1.65	1.59

Таблица 1

соответствии с законом Амдала, на бóльшем числе потоков ускорение от повышения степени параллелизма наблюдается слабо. Особенно от этого страдает MPI реализация.

Скорее всего такие результаты связаны с общей «легкостью» алгоритма с точки зрения вычислений и при этом требовательности к скорости передачи данных.

Исходный код доступен в репозитории: <https://github.com/WoWaster/spbu-hpc>.